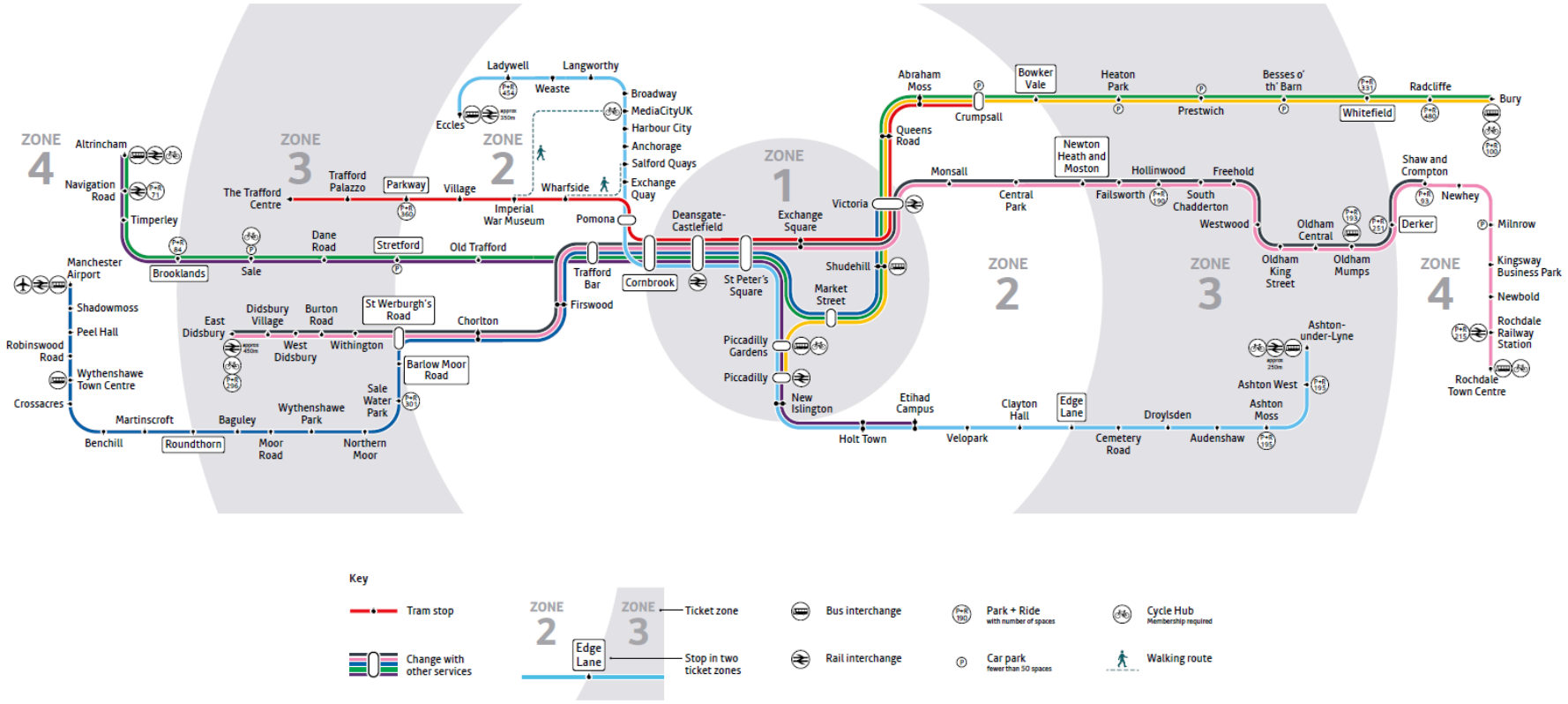
Spatial Join

Claudia Gutiérrez-Arellano

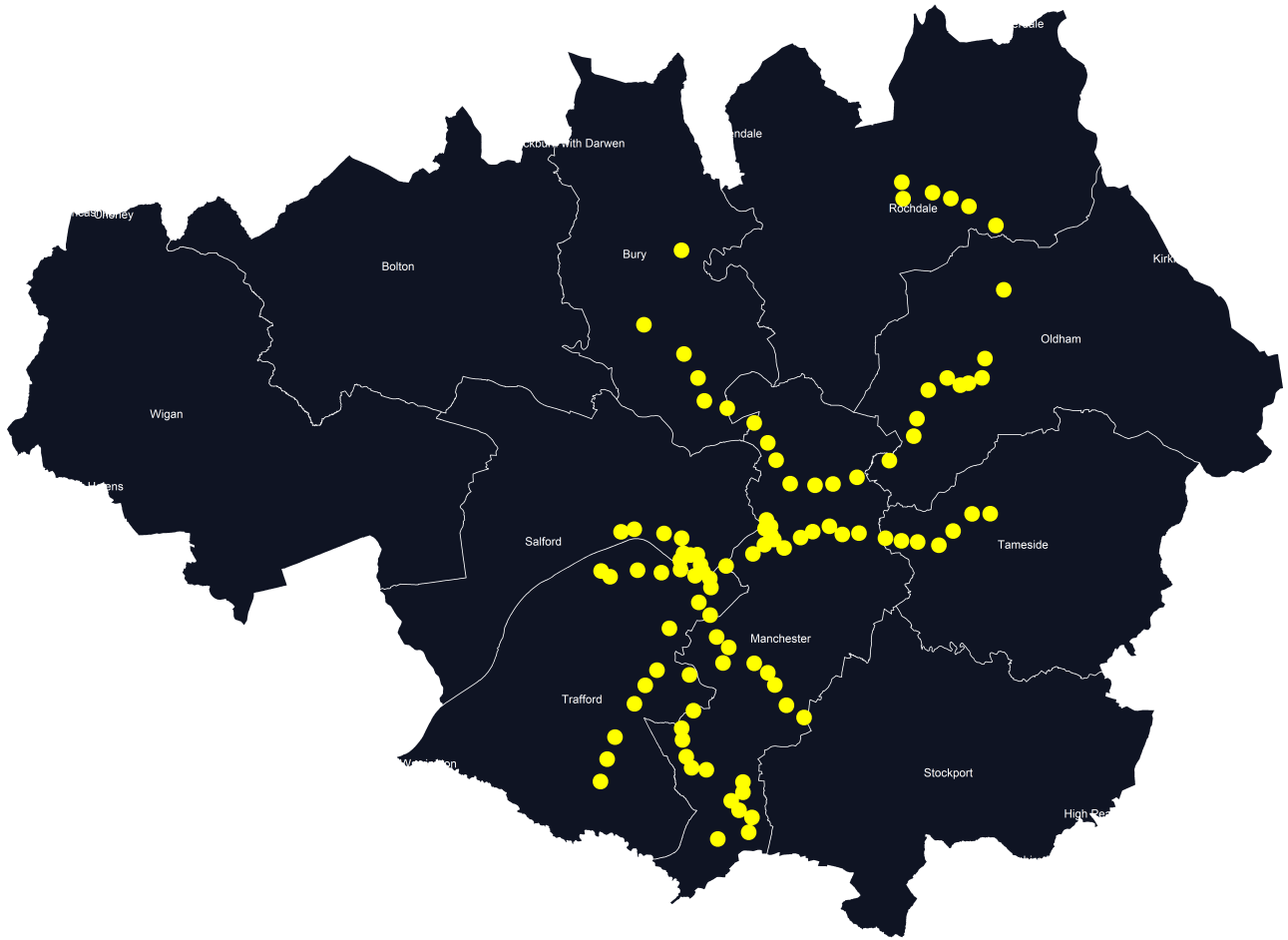
30 April 2026 | The University of Manchester

Which areas in Greater Manchester are best served by the tram network?



Source: TfGM Metrolink Network Map

Tram stops (points) and boroughs (polygons)



GM Metrolink Network, Source: data.gov.uk; Local Authority Districts (Dec 2022), Source: ONS Geography

Spatial join

- Combines attributes from one spatial dataset with another
- **Key feature:** Based on **location**, not shared data

Spatial join

- Combines attributes from one spatial dataset with another
- **Key feature:** Based on **location**, not shared data

- For example:

  Which boroughs are not served by the tram network?

  Which borough has the most tram stops?

  Which tram stops are located within each borough?

Source: Open Street Map, ONS Combined Authorities, TfGM

Spatial join process

Inputs - target & source (same Coordinate Reference System)

- Tram stops as **points**
- Boroughs as **polygons**

Spatial rule

- For each tram stop, identify the borough it falls **within**

Output

- 'Stops in borough' as **points**
  Each tram stop inherits the attributes of a polygon

Spatial join in R

Libraries

```
library(sf)          # simple features
library(tidyverse)   # data management
```

Function: `st_join`

Our Inputs

- target: `stops`
- source: `boroughs`

Our Rule: `st_within` (but there are many more!)

Our Output: `stops_in_borough`

Let's run!

```
stops = st_read("geodata/Metrolink_Stops_Functio
boroughs = st_read("geodata/GM_lad.gpkg", quiet

# check they have the same reference
st_crs(boroughs) == st_crs(stops)
#[1] TRUE

stops_in_borough = st_join(
  stops,
  boroughs,
  join = st_within
)
```

Checking the output

Before:

`stops`

name	stationCode
Heaton Park	HPK
Bowker Vale	BKV
Crumpsall	CRP
Bury	BRY

`boroughs`

LAD22CD	LAD22NM
E06000007	Warrington
E06000008	Blackburn with Darwen
E06000049	Cheshire East
E07000037	High Peak

*Use `glimpse(input_name)` for a quick check

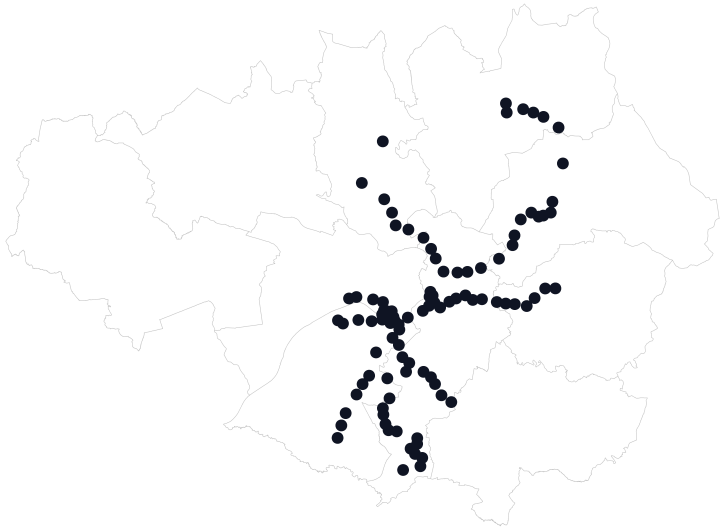
Checking the output

After:

`stops_in_borough`

name	stationCode	LAD22CD	LAD22NM
Heaton Park	HPK	E08000002	Bury
Bowker Vale	BKV	E08000003	Manchester
Crumpsall	CRP	E08000003	Manchester
Bury	BRY	E08000002	Bury

Stops in borough (LAD)



*Use `glimpse(output_name)` for a quick check

Using the joined data to answer questions

How many stops are there in each borough (LADs)?

```
borough_counts = stops_in_borough %>%
  st_drop_geometry() %>%
  count(LAD22NM, name = "n_stops")%>%
  arrange(-n_stops)
```

LAD22NM	n_stops
Manchester	42
Trafford	18
Oldham	10
Salford	10

LAD22NM	n_stops
Tameside	7
Bury	6
Rochdale	6

Which boroughs are not served?

```
not_served = boroughs %>%
  st_drop_geometry() %>%
  anti_join(stops_in_borough, by = "LAD22NM") %>
  select(LAD22NM)
```

LAD22NM
Warrington
Blackburn with Darwen
Cheshire East
High Peak
Chorley

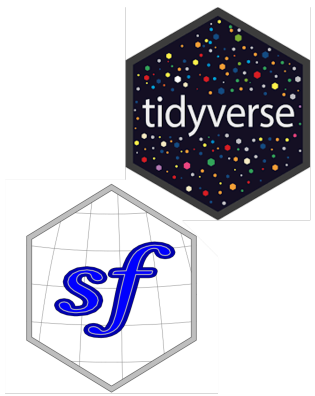
LAD22NM
Rossendale
West Lancashire
Bolton
Stockport

LAD22NM
Wigan
St. Helens
Calderdale
Kirklees

`anti_join` returns all rows from `boroughs` without a match

Spatial join in Python

- Same inputs
- Same spatial rule: within
- Same type of output




```python
import geopandas as gpd

stops_in_borough = gpd.sjoin(
    stops,
    boroughs,
    how="inner",
    predicate="within"
)

borough_counts = (
    stops_in_borough
    .groupby("LAD22NM")
    .size()
    .reset_index(name="n_stops")
)

not_served = (
    boroughs[["LAD22NM"]]
    .merge(stops_in_borough[["LAD22NM"]], on="LA
    .query('_merge == "left_only"')
    [["LAD22NM"]]
)
```

Spatial join in Python

- Same inputs
- Same spatial rule: within
- Same type of output


```python
import geopandas as gpd

stops_in_borough = gpd.sjoin(
    stops,
    boroughs,
    how="inner",
    predicate="within"
)

borough_counts = (
    stops_in_borough
    .groupby("LAD22NM")
    .size()
    .reset_index(name="n_stops")
)

not_served = (
    boroughs[["LAD22NM"]]
    .merge(stops_in_borough[["LAD22NM"]], on="LA
    .query('_merge == "left_only"')
    [["LAD22NM"]]
)
```

Other spatial operations

Spatial relationship	R (sf)	Python (GeoPandas)	Example
point within polygon	`join = st_within`	`predicate="within"`	Which schools are located within flood risk zones?
features overlap	`join = st_intersects`	`predicate="intersects"`	Which green spaces overlap proposed development zones?
boundaries touch	`join = st_touches`	`predicate="touches"`	Which protected areas border urban areas?
nearest feature*	`st_nearest_feature`	`sjoin_nearest()`	Which hospital is closest to a sports facility?

Explore: sf st_join | geopandas sjoin

Summary

- A spatial join links datasets using **location**
- It follows a **process**: inputs > spatial rule > output
- We used a spatial join to identify the boroughs that are/are not served by the tram network
- The same process applies in **R** and **Python**

A question for you

If a borough contains a tram stop, does that mean people across the whole borough have good access to the network?

Not necessarily

- Presence of stops $\neq$ accessibility
- Other analyses (wait to hear about `st_nearest_feature`, `st_distance` and `st_buffer`!)

Questions?

Access R and Python code, data and slides here